Chapter 3 — Digital Electronics

RRB JE CBT-2 · Electronics & Allied Engineering (CEN 05/2025, Annexure-VII C) · Syllabus Unit 3

> **Unit coverage:** Number System & Binary codes · Boolean Algebra & Logic gates · Combinational & Sequential logic circuits · A/D & D/A converters, Counters · Memories

📊 Weightage & Trend Box

Parameter	Detail
Share of technical marks	**≈ 16 %** — the single heaviest technical unit
Questions in 22-04-2025 CBT-2 paper	**8 / 50** technical questions
Difficulty profile	Mostly formula/definition recall + 1-step numericals
Time you should budget	~45–60 s per question

Actual question themes observed in the 22-04-2025 paper:

1. Behaviour of a **priority encoder** (highest-priority input wins).
2. **XOR gate** symbol / gate-combination identification.
3. **Half-adder carry** expression (C = A·B).
4. **T flip-flop** output after 3 clock pulses with T = 1.
5. **2's complement** of 101110.
6. **EX-OR output** when both inputs are 1.
7. **Asynchronous (ripple) counter** — how flip-flops are clocked.
8. **3-to-8 decoder** — number of active outputs at a time.
9. **SRAM vs DRAM** — which needs refreshing / uses capacitor storage.

Preparation strategy: Master (i) complement arithmetic, (ii) all flip-flop truth/excitation tables, (iii) counter MOD logic, (iv) MUX/decoder/encoder behaviour, (v) ADC/DAC resolution formulas, (vi) SRAM–DRAM–ROM comparison. These six blocks alone historically cover 90 % of the questions from this unit.

3.1 Number Systems

A number system is defined by its **radix (base) r** and its digit set {0, 1, …, r−1}. Any number is a weighted sum of powers of the radix:

$N = d_n \cdot r^n + \ldots + d_1 \cdot r^1 + d_0 \cdot r^0 + d_{-1} \cdot r^{-1} + d_{-2} \cdot r^{-2} + \ldots$

System	Base r	Digits	Typical use
Binary	2	0, 1	All digital hardware
Octal	8	0–7	Compact binary (3-bit groups)
Decimal	10	0–9	Human interface
Hexadecimal	16	0–9, A–F	Compact binary (4-bit groups), addresses

With **n bits** you can represent **2^n distinct values**: unsigned range 0 to $2^n - 1$.

3.1.1 Conversions between bases

(a) Binary → Decimal — multiply each bit by its positional weight and add.

$(1\ 0\ 1\ 1\ 0\ 1)_2 = 1{\cdot}32 + 0{\cdot}16 + 1{\cdot}8 + 1{\cdot}4 + 0{\cdot}2 + 1{\cdot}1 = \mathbf{45_{10}}$

(b) Decimal → Binary — repeated division by 2 for the integer part (remainders read bottom-up); repeated multiplication by 2 for the fraction part (integer carries read top-down).

Convert 45_{10}:

```
45 ÷ 2 = 22 rem 1
22 ÷ 2 = 11 rem 0
11 ÷ 2 =  5 rem 1
 5 ÷ 2 =  2 rem 1
 2 ÷ 2 =  1 rem 0
 1 ÷ 2 =  0 rem 1   → read up: 101101₂ ✓
```

Convert 0.625_{10}: 0.625×2 = 1.25 → 1; 0.25×2 = 0.5 → 0; 0.5×2 = 1.0 → 1 ⇒ $\mathbf{0.101_2}$

(c) Binary ↔ Octal — group bits in **threes** from the radix point outward.

$101\ 110_2 \rightarrow 5\ 6 \Rightarrow \mathbf{56_8}$ | Reverse: $7\ 4_8 \rightarrow 111\ 100_2$

(d) Binary ↔ Hexadecimal — group bits in **fours**.

$1011\ 0110_2 \rightarrow B\ 6 \Rightarrow \mathbf{B6_{16}}$ | Reverse: $2F_{16} \rightarrow 0010\ 1111_2$

(e) Octal ↔ Hexadecimal — always go **via binary** (regroup 3-bit ↔ 4-bit).

Example: $375_8 = 011\ 111\ 101_2 = 1111\ 1101_2 = \mathbf{FD_{16}}$

(f) Decimal ↔ any base r — same division/multiplication method with divisor r.

Example: 250_{10} → hex: 250 ÷ 16 = 15 rem 10(A); 15 ÷ 16 = 0 rem 15(F) ⇒ $\mathbf{FA_{16}}$

✍ Worked Example 1 — Multi-base conversion

Convert 156.25_{10} to binary, octal and hexadecimal.

Integer: $156 = 128 + 16 + 8 + 4 = 10011100_2$

Fraction: 0.25 × 2 = 0.5 → 0; 0.5 × 2 = 1.0 → 1 ⇒ $.01_2$

Binary: $\mathbf{10011100.01_2}$

Octal (3-bit groups): 010 011 100 . 010 = $\mathbf{234.2_8}$

Hex (4-bit groups): 1001 1100 . 0100 = $\mathbf{9C.4_{16}}$

3.1.2 Signed number representations

For an n-bit word, three schemes represent negative numbers:

Scheme	−N formed by	Range (n bits)	Zeros	Arithmetic
Sign-magnitude	sign bit 1 + magnitude	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	two (+0, −0)	awkward
1's complement	invert every bit	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	two	end-around carry
2's complement	invert + add 1	**-2^{n-1} … $+(2^{n-1}-1)$**	**one**	**simple — used in all CPUs**

For 8 bits, 2's complement range = **−128 to +127**.

Finding the 2's complement quickly: starting from the LSB, **copy bits up to and including the first 1, then invert all remaining bits**.

✍ Worked Example 2 — 2's complement (2025 PYQ pattern)

Find the 2's complement of 101110.

Method 1: 1's complement = 010001; add 1 → **010010** ✓

Method 2 (shortcut): copy from right up to first 1 → "...10"; invert the rest (1011 → 0100) → **010010** ✓

3.1.3 Complement arithmetic

1's-complement subtraction (A − B): add A to the 1's complement of B; if an **end-around carry** appears, add it back to the LSB (result positive); if no carry, result is negative and in 1's-complement form.

2's-complement subtraction (A − B): add A to the 2's complement of B; **discard** any final carry (positive result); no carry ⇒ negative result in 2's-complement form.

✍ Worked Example 3 — 2's-complement subtraction

Compute $43_{10} - 25_{10}$ using 6-bit 2's-complement arithmetic.

$43 = 101011_2$, $25 = 011001_2$

2's complement of 25 = 100111_2

101011 + 100111 = 1 010010 → discard carry ⇒ **$010010_2 = 18_{10}$** ✓

Compute 25 − 43 (6-bit): 2's comp of 43 = 010101; 011001 + 010101 = 101110 (no carry ⇒ negative). Magnitude = 2's comp of 101110 = 010010 = 18 ⇒ answer **−18** ✓

✍ Worked Example 4 — Overflow detection

Add +90 and +70 in 8-bit 2's complement. Comment.

90 = 01011010, 70 = 01000110. Sum = 10100000 = −96 (wrong sign!).

Two positives yielding a negative ⇒ **overflow** (160 > +127 limit). Rule: overflow occurs when **carry into MSB ≠ carry out of MSB**.

Key facts to memorise

- r's complement = (r−1)'s complement + 1.
- 9's complement of a decimal digit d = 9 − d; 10's complement = 9's complement + 1.
- In 2's complement the MSB has weight **-2^{n-1}** (so 101110_2 as a signed 6-bit number = −32+8+4+2 = −18).

3.2 Binary Codes

A binary **code** assigns bit patterns to symbols. Two families:

- **Weighted codes** — each bit position has a fixed weight: 8421 (natural BCD), 2421, 5211, 84-2-1.
- **Non-weighted codes** — Excess-3, Gray code.

3.2.1 BCD (8421 code)

Each **decimal digit** is coded separately in 4 bits (0000–1001). Codes 1010–1111 are **invalid (6 unused states)**.

Example: 59_{10} = **0101 1001**(BCD) — note this is *not* binary 59 (111011_2).

BCD addition rule: add 4-bit groups; if a group > 1001 **or** produces a carry, **add 0110 (6)** to correct it.

✍ Worked Example 5 — BCD addition

Add 48 + 39 in BCD.

0100 1000 + 0011 1001:

Units: 1000 + 1001 = 1 0001 (carry) → add 0110 → 0111, carry 1 to tens.

Tens: 0100 + 0011 + 1 = 1000 (valid).

Result = 1000 0111 = **87** ✓

3.2.2 Excess-3 (XS-3) code

XS-3 = BCD + 0011. It is **non-weighted** and **self-complementing**: the 9's complement of a digit is obtained by simply **inverting all bits**.

Digit	BCD	XS-3
0	0000	0011
4	0100	0111
5	0101	1000
9	1001	1100

(Check: XS-3 of 4 = 0111; invert → 1000 = XS-3 of 5 = 9−4 ✓.) 2421 and 5211 are self-complementing **weighted** codes.

3.2.3 Gray code (reflected code)

A **unit-distance / cyclic** code — successive values differ in **exactly one bit**. Used in shaft encoders, K-map ordering, asynchronous interfaces (avoids transition glitches).

Binary → Gray: G(MSB) = B(MSB); $G_i = B_{i+1} \oplus B_i$.

Gray → Binary: B(MSB) = G(MSB); $B_i = B_{i+1}$(binary) $\oplus G_i$.

✍ Worked Example 6 — Gray conversions

(a) Binary 1011 → Gray: keep 1; 1⊕0=1; 0⊕1=1; 1⊕1=0 ⇒ **1110**.

(b) Gray 1110 → Binary: keep 1; 1⊕1=0 — careful, XOR previous *binary* bit with current *Gray* bit: B_3=1; B_2=1⊕1=0; B_1=0⊕1=1; B_0=1⊕0=1 ⇒ **1011** ✓ (round-trip consistent).

4-bit Gray sequence begins: 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, …

3.2.4 Alphanumeric codes

- **ASCII** — 7-bit code, 2^7 = **128 characters** (often stored in 8 bits with a parity/0 bit). 'A' = 41_{16} = 65_{10}, 'a' = 61_{16} = 97_{10}, '0' = 30_{16} = 48_{10}.
- **EBCDIC** — 8-bit IBM code (256 symbols).
- **Unicode/UTF-8** — modern multi-byte superset of ASCII.

3.2.5 Error detection & correction

Parity: append one bit so the total number of 1s is even (even parity) or odd (odd parity). Detects any **odd number of bit errors**; cannot correct, cannot detect double errors. A parity generator/checker is a tree of **XOR gates** (XOR of all bits = 1 ⇔ odd number of 1s).

Hamming code: k parity bits protect m data bits when **$2^k \geq m + k + 1$**. Parity bits sit at positions 1, 2, 4, 8, …; the syndrome (recomputed checks) directly gives the **position of a single-bit error**, which is then flipped — Single Error Correction. With one extra overall parity bit → SEC-DED (corrects 1, detects 2).

- For m = 4 data bits ⇒ k = 3 (the classic **Hamming (7,4)** code).
- **Hamming distance** d of a code: detects (d − 1) errors, corrects $\lfloor (d-1)/2 \rfloor$ errors.

✍ Worked Example 7 — Hamming bits

How many parity bits are needed to protect 16 data bits?

Need $2^k \geq 16 + k + 1$. Try k = 5: 32 ≥ 22 ✓ (k = 4 gives 16 ≥ 21 ✗). **Answer: 5 parity bits** (21-bit codeword).

3.3 Boolean Algebra

Boolean algebra (George Boole, 1854; applied to switching by Claude Shannon, 1938) manipulates variables that take only values **0 and 1** with operations AND (·), OR (+), NOT ($\bar{}$).

3.3.1 Postulates and theorems

#	Law	OR form	AND form
1	Identity	$A + 0 = A$	$A \cdot 1 = A$
2	Null (domination)	$A + 1 = 1$	$A \cdot 0 = 0$
3	Idempotent	$A + A = A$	$A \cdot A = A$
4	Complement	$A + \bar{A} = 1$	$A \cdot \bar{A} = 0$
5	Involution	$(\bar{A})\bar{} = A$	—
6	Commutative	$A + B = B + A$	$AB = BA$
7	Associative	$(A+B)+C = A+(B+C)$	$(AB)C = A(BC)$
8	Distributive	$A(B + C) = AB + AC$	**$A + BC = (A+B)(A+C)$**
9	Absorption	$A + AB = A$	$A(A + B) = A$
10	Redundancy	$A + \bar{A}B = A + B$	$A(\bar{A} + B) = AB$
11	Consensus	$AB + \bar{A}C + BC = AB + \bar{A}C$	$(A+B)(\bar{A}+C)(B+C) = (A+B)(\bar{A}+C)$

De Morgan's theorems (the exam favourite):

- $(A + B)\bar{} = \bar{A} \cdot \bar{B}$ — *complement of a sum = product of complements*
- $(A \cdot B)\bar{} = \bar{A} + \bar{B}$ — *complement of a product = sum of complements*

Generalises to any number of variables. Hardware meaning: **NOR = bubbled AND, NAND = bubbled OR**.

Duality principle: in any valid identity, swapping (+ ↔ ·) and (0 ↔ 1) — leaving variables unchanged — yields another valid identity. (The dual of $A + \bar{A}B = A + B$ is $A(\bar{A} + B) = AB$.) The **complement** of a function is found by taking the dual **and** complementing every literal.

✍ Worked Example 8 — Simplification

Simplify $F = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}$.

$= \bar{A}C(\bar{B} + B) + A\bar{B} = \bar{A}C + A\bar{B}$. **$F = \bar{A}C + A\bar{B}$** (3 literals fewer; verify with a truth table if unsure).

3.3.2 Standard forms — SOP, POS, minterms, maxterms

- **Minterm m_i** — an AND term containing **every** variable once (true for exactly one input row). For (A,B,C), $m_5 = A\,\bar{B}\,C$ (101).
- **Maxterm M_i** — an OR term containing every variable once (false for exactly one row). $M_5 = \bar{A} + B + \bar{C}$.
- Relationship: **$M_i = \bar{m}_i$**.
- **Canonical SOP:** F = Σm(list of rows where F = 1). **Canonical POS:** F = ΠM(rows where F = 0). The two lists are complementary: for n variables, Σ-list ∪ Π-list = {0 … 2^n-1}.

Example: F(A,B,C) = Σm(1, 3, 5, 7) = C (all odd rows have C = 1); equivalently F = ΠM(0, 2, 4, 6).

Any function of n variables is one of **$2^{(2^n)}$** possible functions (n = 2 → 16 functions).

3.4 Karnaugh Maps (K-maps)

A K-map is a truth table redrawn so that **adjacent cells differ in one variable** (Gray-code ordering: 00, 01, 11, 10). Grouping adjacent 1s in powers of two (1, 2, 4, 8, …) eliminates the variables that change within the group.

Rules

1. Groups must be rectangles of size 2^k.
2. The map is a **torus** — edges and corners wrap around (the four corner cells of a 4-variable map are adjacent).
3. Make groups **as large as possible**, use **as few groups as possible**; every 1 must be covered; overlap is allowed.
4. A group of 2^k cells in an n-variable map gives a product term of **n − k literals**.
5. **Don't-care (×)** cells may be treated as 1 *or* 0 — include them only when they enlarge a group.
6. Grouping 0s instead gives the minimal **POS** (write the complement, then apply De Morgan).

✍ Worked Example 9 — 4-variable K-map

Minimise F(A,B,C,D) = Σm(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14).

Map (rows AB, columns CD, Gray order 00-01-11-10):

AB\CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	1	1	0	1
10	1	1	0	0

Groups (all cells listed are 1s in the map):

1. **Octet** — columns CD = 00 and CD = 01 (all 8 cells: m0, m1, m4, m5, m12, m13, m8, m9) → variables A, B, D drop out ⇒ term $\bar{C}$.
2. **Quad** — rows AB = 01, 11 × columns CD = 00, 10 (m4, m6, m12, m14) → term $B\bar{D}$. This is what covers m6 and m14 (the C = 1, D = 0 cells of rows 01 and 11).
3. **Quad** — rows AB = 00, 01 × columns CD = 00, 10 (m0, m2, m4, m6) → term $\bar{A}\bar{D}$. This covers the remaining 1 at m2.

Every 1 is now covered (m10 stays 0 and is in no group).

$F = \bar{C} + B\bar{D} + \bar{A}\bar{D}$ — minimal SOP: 3 terms, 5 literals.

✍ Worked Example 10 — Don't-cares (BCD application)

F(A,B,C,D) = Σm(1, 3, 5, 7, 9) + d(10, 11, 12, 13, 14, 15). Minimise.

Map: 1s at all odd minterms 1–9; don't-cares fill the entire invalid-BCD region 10–15.

- Group all cells with D = 1: m1, m3, m5, m7, m9 plus d11, d13, d15 → the full **D column-pair (8 cells)** → term **D**.

F = D — a single literal. (This is exactly the "BCD digit is odd" detector: a BCD number is odd iff its LSB is 1.)

3.5 Logic Gates

3.5.1 The basic and derived gates

Gate	Symbolic expression	Output = 1 when	Truth table (A B → Y)
NOT	$Y = \bar{A}$	A = 0	0→1, 1→0
AND	$Y = A \cdot B$	all inputs 1	00→0, 01→0, 10→0, 11→**1**
OR	$Y = A + B$	any input 1	00→**0**, 01→1, 10→1, 11→1
NAND	$Y = \overline{(AB)}$	any input 0	00→1, 01→1, 10→1, 11→**0**
NOR	$Y = \overline{(A+B)}$	all inputs 0	00→**1**, 01→0, 10→0, 11→0
XOR	$Y = A \oplus B = \bar{A}B + A\bar{B}$	inputs **differ**	00→0, 01→1, 10→1, **11→0**
XNOR	$Y = \overline{(A \oplus B)} = AB + \bar{A}\bar{B}$	inputs **equal**	00→1, 01→0, 10→0, 11→1

XOR properties (exam-critical):

- $A \oplus 0 = A$ (buffer); $A \oplus 1 = \bar{A}$ (controlled inverter); $A \oplus A = 0$; $A \oplus \bar{A} = 1$.
- Both inputs 1 ⇒ output **0** (2025 PYQ).
- Multi-input XOR = 1 when the number of 1-inputs is **odd** → parity element.
- XOR is associative & commutative; used in adders, comparators, Gray converters, parity circuits.
- XNOR is the **equality/coincidence** gate.

Symbols: AND = D-shape; OR = shield/curved shape; XOR = OR with an **extra curved line** on the input side; a **bubble** on any terminal = inversion. (2025 paper asked to identify the XOR symbol among combinations.)

3.5.2 Universal gates — NAND & NOR

NAND and NOR are **universal**: each alone can realise every Boolean function.

All gates from NAND:

Target	NAND realisation	Count
NOT	A NAND A	1
AND	(A NAND B) then NOT	2
OR	(A NAND A) NAND (B NAND B) — De Morgan	3
NOR	OR then NOT	4
XOR	standard 4-gate network: Y = A NAND (A NAND B), B NAND (A NAND B), then NAND together	**4**
XNOR	XOR + inverter	5

All gates from NOR:

Target	NOR realisation	Count
NOT	A NOR A	1
OR	(A NOR B) then NOT	2
AND	(A NOR A) NOR (B NOR B)	3
NAND	AND then NOT	4
XNOR	4-gate NOR network	4
XOR	XNOR + inverter	5

Bubbled-gate (De Morgan) equivalences — remember as pictures:

- **Bubbled-input AND ≡ NOR** ($\bar{A}\cdot\bar{B} = \overline{(A+B)}$)
- **Bubbled-input OR ≡ NAND** ($\bar{A}+\bar{B} = \overline{(AB)}$)
- Bubbled-input NAND ≡ OR; bubbled-input NOR ≡ AND.
- Two cascaded inverting bubbles cancel — the basis of **NAND-NAND = AND-OR (SOP)** and **NOR-NOR = OR-AND (POS)** two-level realisations.

3.5.3 Common 7400-series IC numbers (frequent one-mark questions)

IC	Function	IC	Function
7400	Quad 2-input **NAND**	7447	BCD → 7-segment decoder/driver (common-anode)
7402	Quad 2-input **NOR**	7474	Dual **D flip-flop** (+ve edge)
7404	Hex **inverter** (NOT)	7476	Dual **JK flip-flop**
7408	Quad 2-input **AND**	7483	4-bit **binary full adder**
7432	Quad 2-input **OR**	7485	4-bit magnitude **comparator**
7486	Quad 2-input **XOR**	7490	**Decade counter** (mod-10)
7410	Triple 3-input NAND	7493	4-bit **ripple counter** (mod-16)
74138	**3-to-8 decoder**	74151	**8:1 MUX**
74147	Decimal **priority encoder** (10-to-4)	74148	Octal **priority encoder** (8-to-3)
74194	4-bit **universal shift register**	555	Timer (clock source; not TTL logic)

3.5.4 Logic families — TTL, CMOS, ECL

Parameter	TTL (74xx)	CMOS (40xx/74HC)	ECL (10K)
Basic element	Bipolar (multi-emitter BJT)	Complementary MOSFET pair	BJT differential pair (non-saturating)
Supply	+5 V fixed	3–15 V (4000 series)	−5.2 V
Propagation delay	~10 ns (74LS)	~50 ns (4000) / ~10 ns (74HC)	**~1–2 ns (fastest)**
Static power/gate	~10 mW (74) / 2 mW (LS)	**~nW–µW (lowest static)**	~25–50 mW (highest)
Fan-out	~10	**≥ 50** (very high, capacitive-limited)	~25
Noise margin	~0.4 V	**~30–45 % of V_DD (best)**	~0.2–0.3 V (worst)
Noise immunity	moderate	**highest**	lowest
Logic swing	0.2 V–3.4 V approx.	rail-to-rail	small (~0.8 V)
Saturation?	Transistors saturate	—	**Never saturates → speed**
Remark	Rugged workhorse	Power-critical & VLSI standard	Supercomputers, RF prescalers

TTL levels (5 V): $V_{IL(max)} = 0.8$ V, $V_{IH(min)} = 2.0$ V, $V_{OL(max)} = 0.4$ V, $V_{OH(min)} = 2.4$ V ⇒ noise margins $NM_L = NM_H = 0.4$ V.

Other memorables:

- **Figure of merit = speed–power product** (pJ); lower is better. 74LS ≈ 2 mW × 10 ns = 20 pJ.
- CMOS dynamic power = $C \cdot V^2 \cdot f$ — rises with frequency; at high f CMOS can exceed TTL power.
- **Fan-in** = number of inputs a gate can accept; **fan-out** = number of standard loads an output can drive.
- Unused CMOS inputs must **never float** (tie to rail); floating TTL inputs read as logic 1 (bad practice but common in questions).
- **Totem-pole outputs must not be wire-ANDed**; use **open-collector** (needs external pull-up; allows wired-AND) or **tri-state** (0, 1, Hi-Z; allows bus sharing).

3.6 Combinational Logic Circuits

Output depends **only on present inputs** — no memory, no feedback, no clock. Design flow: truth table → K-map minimisation → gate realisation.

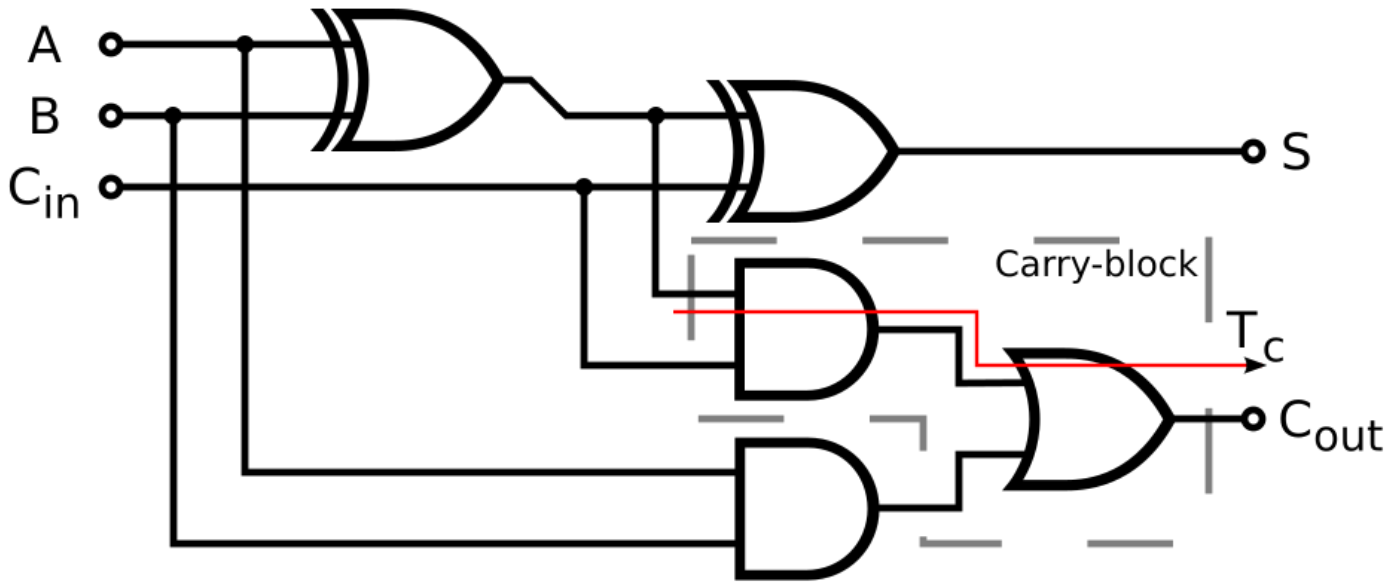
3.6.1 Half adder (HA)

Adds two bits A, B → Sum S, Carry C.

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$S = A \oplus B$ $C = A \cdot B$ (2025 PYQ: carry = AND of inputs). Hardware: **1 XOR + 1 AND**.

3.6.2 Full adder (FA)



Full-adder logic diagram.

Adds A, B and carry-in Cin → S, Cout.

S = A ⊕ B ⊕ Cin Cout = AB + Cin(A ⊕ B) = AB + BCin + ACin (majority function)

- Built from **2 half adders + 1 OR gate**.
- A full adder needs **min. 2-level logic**; sum is odd-parity of the three inputs.

3.6.3 Half & full subtractor

Circuit	Difference	Borrow
Half subtractor (A − B)	D = A ⊕ B	B_out = **$\bar{A}\cdot B$**
Full subtractor (A − B − Bin)	D = A ⊕ B ⊕ Bin	B_out = $\bar{A}B$ + Bin$\overline{(A \oplus B)}$ = $\bar{A}B$ + $\bar{A}$Bin + B·Bin

Note the symmetry: subtractor = adder with the minuend **complemented in the borrow term**. In practice subtraction is done by **adding the 2's complement** (adder + XOR-controlled inverters + Cin = 1 → a single adder/subtractor unit with mode line M: B-inputs pass through XOR gates with M; M = 0 add, M = 1 subtract).

3.6.4 Parallel (ripple-carry) adder & look-ahead

- **n-bit ripple-carry adder** = n full adders, carry ripples; worst-case delay ∝ n (≈ 2n gate delays).
- **Carry-look-ahead (CLA)** computes all carries in parallel using generate **$G_i = A_iB_i$** and propagate **$P_i = A_i \oplus B_i$**: $C_{i+1} = G_i + P_iC_i$ expanded — delay becomes ~constant (4 gate levels). IC 7483 is a 4-bit adder with internal look-ahead.

3.6.5 Magnitude comparator

Compares two n-bit words → three outputs A>B, A=B, A<B.

- 1-bit: (A>B) = $A\bar{B}$; (A<B) = $\bar{A}B$; (A=B) = $A \odot B$ (XNOR).
- n-bit equality = AND of all bitwise XNORs.
- **IC 7485** = 4-bit comparator, cascadable via expansion inputs.

3.6.6 Multiplexer (MUX) — "many to one"

A 2^n:1 MUX has 2^n data inputs, **n select lines**, 1 output: Y = D_s (the selected input). It is a digitally controlled switch / **data selector**; also called a universal logic element.

- 2:1 MUX: $Y = \bar{S} \cdot D_0 + S \cdot D_1$.
- 4:1 MUX: $Y = \bar{S}_1\bar{S}_0D_0 + \bar{S}_1S_0D_1 + S_1\bar{S}_0D_2 + S_1S_0D_3$.
- 8:1 MUX = IC 74151. Larger MUX from smaller: an 8:1 needs two 4:1 + one 2:1 (or a tree).
- Number of select lines for an N:1 MUX = $\mathbf{\log_2 N}$.

Implementing any n-variable function with a 2^n:1 MUX: connect the n variables to the select lines and hard-wire each data input D_i to the function value (0/1) of minterm i.

Better — 2^{n-1}:1 MUX method: use n−1 variables as selects; each data input then gets one of {0, 1, remaining variable, its complement} read off the folded truth table.

✍ Worked Example 11 — Function on a 4:1 MUX

Implement F(A,B,C) = Σm(1, 2, 6, 7) using a 4:1 MUX with A, B as select lines (S_1 = A, S_0 = B).

Partition minterms by AB:

AB	minterms present	C-column	D_i
00	m1 (C=1) ✓, m0 ✗	F = C	**$D_0 = C$**
01	m2 (C=0) ✓, m3 ✗	F = $\bar{C}$	**$D_1 = \bar{C}$**
10	none (m4, m5 absent)	F = 0	**$D_2 = 0$**
11	m6, m7 both ✓	F = 1	**$D_3 = 1$**

One 4:1 MUX + one inverter realises F. ✓

3.6.7 Demultiplexer (DEMUX) — "one to many"

1 input, n select lines, 2^n outputs; input is routed to the selected output, all others 0. A **DEMUX = decoder with an enable used as the data input**. 1:8 DEMUX ≡ 3-to-8 decoder (74138) with data on $\bar{E}$.

3.6.8 Decoder

An **n-to-2^n decoder** activates **exactly one** output line — the one whose index equals the binary input (2025 PYQ: a 3-to-8 decoder has **only 1 active output at any time**; the other 7 stay inactive). Active-low versions (74138) pull the selected line LOW.

- 2-to-4: 4 AND gates, outputs = the 4 minterms of (A, B).
- **3-to-8 (74138):** outputs = all 8 minterms; has enables (G1, $\bar{G}$2A, $\bar{G}$2B) for cascading — two 74138s + inverter = 4-to-16 decoder.
- **Decoder + OR gates implements any function** (OR the required minterm outputs) — this is how ROMs work.
- **BCD-to-7-segment** (7447/7448) is a code-converter decoder.

3.6.9 Encoder & priority encoder

Encoder = inverse of decoder: 2^n inputs → n-bit binary code of the active input. Simple encoders fail if **two inputs are active simultaneously** (outputs OR together, giving a wrong code) and cannot distinguish "input 0 active" from "no input".

Priority encoder fixes this: when several inputs are active, the output encodes the **highest-priority (highest-numbered) active input**; lower inputs are ignored (2025 PYQ). A **valid/GS output** flags that at least one input is active.

- 74148 = 8-to-3 priority encoder; 74147 = decimal (10-to-4, BCD out) priority encoder.
- Example: inputs I_6 and I_3 both high on a 74148-type encoder → output = **110 (6)**.

3.6.10 Code converters & parity circuits

- **Binary → Gray:** MSB straight, then chain of XORs ($G_i = B_{i+1} \oplus B_i$) — n-bit needs **n−1 XOR gates**.
- **Gray → Binary:** cascaded XORs with feedback of computed binary bits (n−1 XORs, but ripple delay).
- **BCD ↔ XS-3:** add/subtract 0011 with a 4-bit adder or direct K-map design.
- **Even-parity generator** for n bits: XOR tree of all n bits; **checker:** XOR of (n data + parity) — output 1 ⇒ error (odd count).

3.7 Sequential Logic Circuits

Output depends on present inputs **and past history** — the circuit has **memory** (feedback). Classes:

- **Asynchronous** (level/unclocked — latches, ripple counters) vs **Synchronous** (clock-controlled — flip-flops, synchronous counters, registers).

3.7.1 Latch vs flip-flop; triggering

Feature	Latch	Flip-flop
Sensitivity	**Level**-sensitive (transparent while enable active)	**Edge**-triggered (samples only at clock edge)
Clock	Optional enable	Required
Race problems	Transparent — glitches pass	Immune between edges
Example	SR latch (NOR/NAND), gated D latch	7474 D-FF, 7476 JK-FF

Triggering types: positive (rising) edge ↑, negative (falling) edge ↓ (bubble on the clock pin), pulse-triggered (master–slave).
Asynchronous inputs PRESET (Q→1) and CLEAR (Q→0) override the clock.

3.7.2 SR latch / SR flip-flop

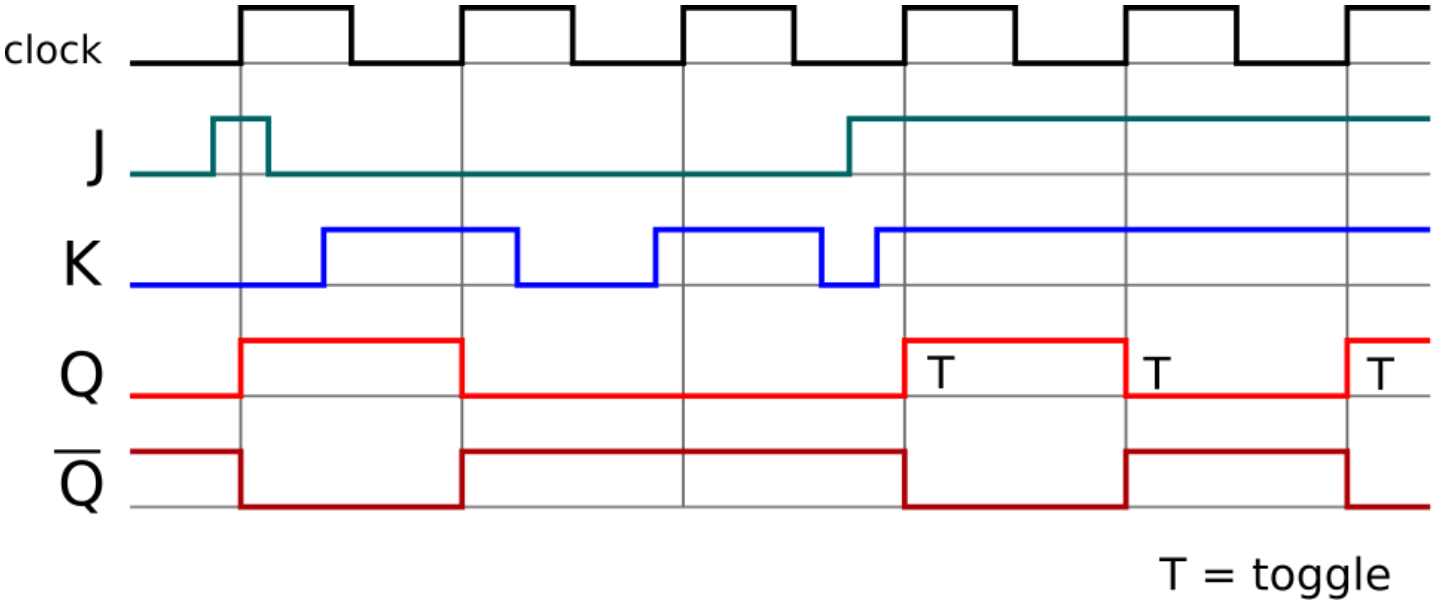
NOR-based SR latch (active-high S, R):

S	R	Q(next)	State
0	0	Q (hold)	Memory
0	1	0	Reset
1	0	1	Set
1	1	**Invalid** (Q = $\bar{Q}$ = 0; race on release)	Forbidden

NAND-based ($\bar{S}\bar{R}$) latch is active-low: 11 = hold, **00 = invalid**.

Characteristic equation: $\mathbf{Q^+ = S + \bar{R}Q}$ (with SR = 0 constraint).

3.7.3 JK flip-flop



JK flip-flop timing diagram.

Removes the SR forbidden state: J=K=1 **toggles**.

J	K	Q^+	Action
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}$	Toggle

Characteristic equation: $\mathbf{Q^+ = J\bar{Q} + \bar{K}Q}$.

Race-around condition: in a level-clocked JK with J=K=1, if clock pulse-width t_p > propagation delay t_pd, the output toggles repeatedly during one pulse — final state unpredictable. **Cures:** (i) make t_p < t_pd (impractical), (ii) **edge triggering**, (iii) **master–slave JK**: master latch accepts data while CLK = 1, slave copies it when CLK = 0 → exactly one toggle per pulse ("pulse-triggered"; data changes appear at output on the falling edge). Master–slave JKs can still suffer *ones-catching*; modern parts are pure edge-triggered.

3.7.4 D (data/delay) flip-flop

$Q^+ = \mathbf{D}$ — output follows D at each clock edge (one-clock delay). Built from JK with **$J = D, K = \bar{D}$**, or SR with S = D, R = $\bar{D}$. Used in registers, pipelines, synchronisers. A **gated D latch** (74LS75) is the level-sensitive cousin.

3.7.5 T (toggle) flip-flop

$Q^+ = \mathbf{T \oplus Q}$.

T	Action
0	Hold ($Q^+ = Q$)
1	**Toggle** ($Q^+ = \bar{Q}$)

Built from JK with **$J = K = T$**, or from D with $D = T \oplus Q$. With T = 1 the output is a square wave at **$f_{clk} / 2$** — the frequency-divider element of every binary counter.

2025 PYQ logic: T = 1, $Q_0 = 0$. After clock 1 → Q = 1; clock 2 → Q = 0; clock 3 → **Q = 1** (odd number of toggles inverts the initial state).

3.7.6 Truth (characteristic) & excitation tables — memorise cold

Characteristic equations

FF	Q^+
SR	$S + \bar{R}Q$ ($SR = 0$)
JK	$J\bar{Q} + \bar{K}Q$
D	D
T	$T \oplus Q$

Excitation tables (what inputs produce a desired $Q \rightarrow Q^+$ transition; × = don't care):

$Q \rightarrow Q^+$	S R	J K	D	T
0 → 0	0 ×	0 ×	0	0
0 → 1	1 0	1 ×	1	1
1 → 0	0 1	× 1	0	1
1 → 1	× 0	× 0	1	0

3.7.7 Flip-flop conversions (design with excitation tables)

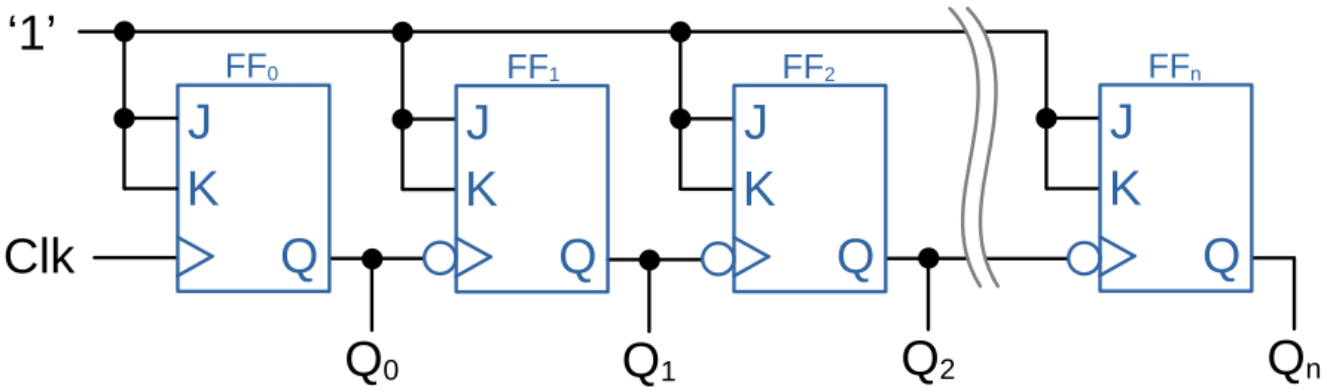
From → To	Required input logic
JK → D	$J = D,\ K = \bar{D}$
JK → T	$J = K = T$
JK → SR	$J = S,\ K = R$
D → T	$D = T \oplus Q$
D → JK	$D = J\bar{Q} + \bar{K}Q$
T → D	$T = D \oplus Q$
T → JK	$T = J\bar{Q} + KQ$
SR → JK	$S = J\bar{Q},\ R = KQ$
SR → D	$S = D,\ R = \bar{D}$
SR → T	$S = T\bar{Q},\ R = TQ$

Method (works for any pair): write target FF's truth table → for each row use the *source* FF's excitation table → K-map the source inputs.

3.8 Counters

A counter is a register that steps through a fixed sequence of states on clock pulses. **MOD number = number of distinct states**. With n flip-flops, MOD ≤ 2^n, and the required n for MOD-N is **$n = \lceil \log_2 N \rceil$**.

3.8.1 Asynchronous (ripple) counter



Asynchronous (ripple) counter using JK flip-flops.

- **Only the first flip-flop receives the external clock; each subsequent FF is clocked by the output of the previous stage** (2025 PYQ — this is the defining feature).
- All FFs in toggle mode (T = 1 or JK with J = K = 1).
- **Up counter:** negative-edge FFs clocked from **Q** of the previous stage (or +ve edge from $\bar{Q}$).
- **Down counter:** clock from **$\bar{Q}$** (with −ve edge FFs) — or read outputs from $\bar{Q}$ of an up counter.
- Each stage divides frequency by 2: after n stages, **$f_{out} = f_{in} / 2^n$**.

- **Drawback — ripple delay:** total settling time = n·t_pd; decoded outputs glitch; maximum clock frequency **f_max = 1 / (n·t_pd)**.

Mod-N ripple counter (e.g. decade/7490): detect the first unwanted count with a NAND gate and drive asynchronous CLEAR. Decade (mod-10) counter: NAND of Q_3 and Q_1 fires at count 1010 → resets to 0000 → counts 0–9. Cascading a mod-10 and a mod-6 gives mod-60 (digital clocks).

✍ Worked Example 12 — Ripple counter speed & state

(a) A 4-bit ripple counter uses FFs with t_pd = 25 ns. Maximum clock frequency?

f_max = 1/(n·t_pd) = 1/(4 × 25 ns) = **10 MHz**.

(b) Starting from 0000, what does the counter read after 75 input pulses?

75 mod 16 = 11 → **1011_2**.

(c) Frequency at the MSB output for a 1 MHz clock? 1 MHz / 2^4 = **62.5 kHz**.

3.8.2 Synchronous counters

All flip-flops share the common clock; combinational logic on J/K (or T) inputs decides which FFs toggle. Standard binary up-counter condition: stage i toggles when **all lower stages are 1**: $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_0Q_1$, $T_3 = Q_0Q_1Q_2$.

- No cumulative delay → **f_max = 1/(t_pd + t_gate)** regardless of length; glitch-free decoding.
- Design procedure for arbitrary sequences: state diagram → state table → excitation table → K-maps → logic. Unused states must be checked for **lock-out** (design them to re-enter the main sequence = self-correcting counter).
- **Up/down counter:** mode input selects whether stage enables come from Q (up) or $\bar{Q}$ (down) — IC 74192/74193.

3.8.3 Ring counter

An n-bit shift register with **Q of the last stage fed back to D of the first**, initialised with a single 1 (e.g. 1000).

- **MOD = n** (n states) — one FF per state, outputs are already decoded (one-hot).
- Sequence for n = 4: 1000 → 0100 → 0010 → 0001 → 1000 …
- Output frequency of each Q = f_clk / n; duty cycle 1/n.

3.8.4 Johnson (twisted-ring / switch-tail) counter

Feedback taken from **$\bar{Q}$ of the last stage** to D of the first.

- **MOD = 2n** with n flip-flops.
- n = 4 sequence: 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → repeat (8 states).
- Decoding any state needs only a **2-input AND gate**; used for quadrature/multi-phase clocks. Must be protected against the 2^n − 2n unused states.

3.8.5 Counter summary table

Counter (n FFs)	Clocking	MOD	Max freq	Decoding
Ripple (binary)	serial (Q → next CLK)	2^n	1/(n·t_pd) — slowest	glitchy
Synchronous binary	common clock	2^n	1/(t_pd + t_gate) — fast	clean
Ring	common clock (shift)	**n**	fast	none needed (one-hot)
Johnson	common clock (shift)	**2n**	fast	2-input gate

MOD-count quick answers: 3 FF ripple → MOD-8, ÷8 frequency. MOD-10 (decade) needs 4 FFs. MOD-60 = MOD-10 × MOD-6. To divide a clock by 32 → 5 T-FFs in ripple.

3.9 Shift Registers

A cascade of D flip-flops sharing a clock; data moves one stage per clock.

Type	Load	Read	Clocks to move n bits in / out
SISO	serial	serial	n in, n out (n-clock delay line)
SIPO	serial	parallel	n in, 0 (instant read)
PISO	parallel (1 clk)	serial	1 in, n out — parallel→serial converter
PIPO	parallel	parallel	1 and 1 — storage register

- **Universal shift register (74194/74195):** mode controls S_1S_0 select hold / shift-right / shift-left / parallel-load.
- Applications: serial↔parallel conversion, delay lines (SISO delay = n/f_clk), multiplication/division by 2 (left/right shift), ring & Johnson counters, PRBS generators (LFSR).
- Time to fully load an 8-bit SISO register at 1 MHz = 8 µs.

3.10 D/A Converters (DAC)

A DAC converts an n-bit digital word D into an analog output $V_{out} = K \cdot V_{ref} \cdot (D/2^n)$.

Key formulas

- **Resolution (step size) = $V_{FS} / (2^n - 1)$** volts per LSB (V_{FS} = full-scale output). As a fraction/percentage: $1/(2^n - 1) \times 100$ %.
- Number of levels = 2^n; number of steps = $2^n - 1$.
- Analog output for input code D: **$V_{out} = D \times$ step size**.
- Specs: resolution, accuracy, linearity (±½ LSB), monotonicity, settling time, glitch energy.

3.10.1 Weighted-resistor DAC

A summing op-amp whose input resistors are **binary-weighted**: the MSB uses R, the next bit 2R, then 4R, 8R, … (bit of weight 2^i gets resistance halved relative to the next lower bit, so its current is doubled). Output $V_o = -R_f \cdot \Sigma\ (b_i \cdot V_{ref} / R_i)$, i.e. each 1-bit injects a current proportional to its binary weight.

Drawback: an n-bit DAC needs a resistor spread of $2^{n-1} : 1$ (8-bit → 128:1) with matched precision — impractical beyond ~6 bits.

3.10.2 R-2R ladder DAC

Uses **only two resistor values (R and 2R)** regardless of word length — easy to fabricate and match; the industry standard.

$V_{out} = V_{ref} \times D / 2^n$ (inverted-ladder form, unity gain), where D = decimal value of the input word. Each node of the ladder divides by 2 (Thevenin resistance looking into any node = R).

✍ Worked Example 13 — DAC numericals

(a) A 4-bit R-2R DAC has V_{ref} = 16 V. Output for input 1011?

$V_{out} = 16 \times 11/2^4 = 16 \times 11/16 =$ **11 V**.

(b) An 8-bit DAC has full-scale 10 V. Resolution?

Step = $10/(2^8 - 1) = 10/255 =$ **39.2 mV**; % resolution = 1/255 = **0.392 %**.

(c) How many bits for a resolution better than 5 mV on a 10 V range?

$10/(2^n - 1) \le 5$ mV → $2^n - 1 \ge 2000$ → n = 11 (2047 ✓). **11 bits**.

3.11 A/D Converters (ADC)

An ADC samples an analog voltage and outputs an n-bit code. Two-step process: **sample-and-hold**, then **quantise + encode**.

Key formulas

- **Resolution = $V_{FS} / 2^n$** (quantum Q); **quantisation error = ±½ LSB = Q/2** max.
- Dynamic range ≈ 6.02 n dB; SNR (ideal) = 6.02 n + 1.76 dB.
- Nyquist: $f_{sample} \ge 2 f_{max(signal)}$.

3.11.1 Flash (parallel/simultaneous) ADC

- **$2^n - 1$ comparators** + resistor ladder + priority encoder.
- **Fastest** ADC (one comparator delay, ~ns) — video/oscilloscope use.
- Hardware doubles per extra bit → practical only to ~8–10 bits. (3-bit flash → 7 comparators; 8-bit → 255.)

3.11.2 Counter (ramp) ADC

Counter drives a DAC; comparator stops the count when $V_{DAC} \ge V_{in}$.

- Simple; conversion time **variable, up to $2^n - 1$ clocks** (worst case) → slow.
- Tracking (up/down) ADC is the continuous version.

3.11.3 Successive-approximation ADC (SAR)

Binary search: SAR sets each bit MSB→LSB, DAC output compared with V_{in}, bit kept or cleared.

- **Conversion time = n clock cycles — fixed, independent of V_in.**
- Best speed/complexity/cost trade-off → the most widely used general-purpose ADC (8–16 bits). Needs S/H at input.

3.11.4 Dual-slope (integrating) ADC

Integrate V_in for fixed time T_1, then de-integrate with −V_ref and count time T_2: **$V_{in} = V_{ref} \cdot T_2/T_1$** (counts ∝ V_in).

- **Highest accuracy & noise rejection** — errors in R, C, clock cancel (ratiometric).
- Choosing T_1 = k/50 Hz rejects mains hum (excellent NMR).
- Very **slow** (ms) → digital multimeters, panel meters.

3.11.5 ADC comparison table

Type	Conversion time	Hardware	Accuracy	Typical use
Flash	~1 comparator delay (fastest)	2^n-1 comparators (huge)	moderate	video, DSO
SAR	n clocks (fixed)	1 DAC + SAR (moderate)	good	data acquisition, µC
Counter/ramp	≤ 2^n-1 clocks (variable)	minimal	moderate	cheap/slow apps
Dual-slope	2 integration periods (slowest)	integrator + counter	**best**	DMM, instrumentation

✍ Worked Example 14 — ADC numericals

(a) A 10-bit ADC spans 0–10 V. Resolution and max quantisation error?

Q = $10/2^{10}$ = 10/1024 = **9.77 mV**; error = ±Q/2 = **±4.88 mV**.

(b) An 8-bit SAR ADC uses a 1 MHz clock. Conversion time?

t = n/f = 8 × 1 µs = **8 µs** (same for every input voltage).

(c) Comparators in a 6-bit flash ADC? $2^6 - 1$ = **63**.

(d) A 12-bit counter ADC with 1 MHz clock — worst-case conversion time?

($2^{12} - 1$) × 1 µs = 4095 µs ≈ **4.1 ms**.

3.12 Memories

A memory stores binary words. Organisation **M × N** = M words (addresses) × N bits per word; address lines k where **$2^k = M$**; capacity = M × N bits.

3.12.1 Classification

```
Semiconductor memory
├── Volatile — RAM (read/write; loses data at power-off)
│   ├── SRAM  (static)
│   └── DRAM  (dynamic)
└── Non-volatile — ROM family (retains data)
    ├── Mask ROM  (factory-programmed)
    ├── PROM      (one-time, fuse-blown)
    ├── EPROM     (erase: UV light, whole chip; quartz window)
    ├── EEPROM    (erase: electrical, byte-wise)
    └── Flash     (electrical, block/sector-wise; SSDs, pen drives)
```

3.12.2 SRAM vs DRAM (2025 PYQ)

Parameter	SRAM	DRAM
Storage element	**Cross-coupled latch (6 transistors)**	**1 transistor + 1 capacitor**
Refresh needed?	**No** — data held while powered	**Yes** — capacitor leaks; refresh every ~2–64 ms
Speed	Faster (sub-10 ns access)	Slower
Density / cost per bit	Low density, expensive	**High density, cheap**
Power (standby)	Lower	Higher (refresh)
Interface	Simple	Multiplexed row/column address (RAS/CAS)
Use	**Cache memory**, registers	**Main memory**

Both are **volatile**. Exam one-liners: "*Which memory must be refreshed periodically?*" → *DRAM*. "*Which stores data in a capacitor?*" → *DRAM*. "*Which is used for cache?*" → *SRAM*.

3.12.3 ROM family details

Type	Programmed by	Erased by	Granularity
Mask ROM	manufacturer mask	never	—
PROM	user, once (fusible links)	never	—
EPROM	user (high-V pulses)	**UV light ~20 min**	entire chip
EEPROM	user, in-circuit	electrically	**byte**
Flash	user, in-circuit	electrically	**block/sector** (fast bulk erase)

- A ROM = fixed **decoder (AND array) + programmable OR array** → can realise any combinational function (lookup table). Related PLDs: **PROM** (fixed AND, prog. OR), **PAL** (prog. AND, fixed OR), **PLA** (both programmable — most flexible).
- Flash uses **floating-gate MOSFETs**; NAND flash = storage (SSD), NOR flash = code execution.

3.12.4 Memory expansion & address decoding

- **Address lines for M words: k = $\log_2 M$.** 1K = 2^{10} = 1024, 1M = 2^{20}, 1G = 2^{30}.
- **Word-length expansion:** place chips side-by-side, same addresses, data buses concatenated.

- **Word-capacity expansion:** stack chips; extra high-order address bits feed a **decoder driving chip-selects (CS)**.

✍ Worked Example 15 — Memory calculations

(a) Address & data lines for a 4K × 8 memory?

4K = 2^{12} ⇒ **12 address lines, 8 data lines**; capacity = 4096 × 8 = 32 768 bits = 4 KB.

(b) How many 1K × 4 chips build a 4K × 8 memory?

Words: 4K/1K = 4 rows; width: 8/4 = 2 per row ⇒ 4 × 2 = **8 chips**, with a **2-to-4 decoder** on $A_{11}A_{10}$ for chip-select.

(c) A ROM realising any 4-input, 3-output function needs $2^4 \times 3$ = **48 bits** (16 words × 3).

(d) Address range of a 16K block starting at 2000_{16}? 16K = 4000_{16} locations → 2000_{16} to **$5FFF_{16}$**.

3.12.5 Memory hierarchy basics

Registers → **Cache (SRAM)** → Main memory (DRAM) → Secondary (Flash/disk). Moving down: size ↑, cost/bit ↓, speed ↓. Cache exploits **locality of reference**; hit ratio h gives average access time t_avg = h·t_cache + (1−h)·t_main.

3.13 PYQ Corner — the 9 observed questions of 22-04-2025, solved

Q1. In a priority encoder, if two inputs are active simultaneously, the output corresponds to —

The input with the **higher priority** (in 74147/74148, the higher-numbered input); the lower one is ignored. E.g. I_5 and I_2 both high → code for 5. ✓

Q2. Which combination of gates gives an output identical to an XOR gate?

$Y = \bar{A}B + A\bar{B}$ — i.e. **two AND gates + one OR gate + two inverters** (or the 4-NAND network). Recognise the symbol: OR shape with an extra curved input line. Also XOR = $(A + B)\cdot\overline{(AB)}$ — an OR and a NAND feeding an AND. ✓

Q3. The carry output of a half adder is —

C = A · B (AND of the two inputs); Sum = A ⊕ B. ✓

Q4. A T flip-flop has T = 1 and starts with Q = 0. Output after 3 clock pulses?

Toggles every clock: 0 → 1 → 0 → **1**. Answer: **Q = 1** (odd number of pulses inverts the start state; after even pulses it returns to 0). ✓

Q5. The 2's complement of 101110 is —

1's complement 010001 + 1 = **010010**. (Shortcut: copy trailing "10", invert the rest.) ✓

Q6. Output of an EX-OR gate when both inputs are 1 —

1 ⊕ 1 = **0**. (XOR is 1 only for *different* inputs; a multi-input XOR outputs 1 for an odd number of 1s.) ✓

Q7. In an asynchronous (ripple) counter, the flip-flops are clocked —

Only the first FF gets the external clock; every following FF is clocked by the output (Q or $\bar{Q}$) of the preceding FF — hence the count "ripples" through with cumulative delay n·t_pd. (Contrast: synchronous counter — one common clock to all FFs.) ✓

Q8. In a 3-to-8 decoder, how many outputs are active at a time?

Exactly one — the output whose index equals the 3-bit input; the remaining seven are inactive. (Active-low in the 74138: the selected line goes LOW.) ✓

Q9. Which RAM stores data as charge on a capacitor and therefore needs periodic refreshing?

DRAM (1T-1C cell, refresh every few ms). SRAM uses a 6-transistor latch, needs no refresh, is faster, and serves as cache. ✓

3.14 Practice MCQs (30 questions)

1. $(2A9)_{16}$ in decimal is —

(a) 649 (b) 681 (c) 665 (d) 425

2. The octal equivalent of binary 110101011 is —

(a) 653 (b) 565 (c) 635 (d) 6AB

3. The range of signed 2's-complement numbers with 8 bits is —

(a) −127 to +127 (b) −128 to +127 (c) −128 to +128 (d) 0 to 255

4. The 1's complement of 0110 is —

(a) 1010 (b) 1001 (c) 0101 (d) 1110

5. Using 2's-complement addition, 0101 − 0011 (4-bit) equals —

(a) 0010 (b) 0100 (c) 1000 (d) 0001

6. Which code is both weighted and self-complementing?

(a) 8421 (b) Excess-3 (c) 2421 (d) Gray

7. The Gray code for binary 1101 is —

(a) 1011 (b) 1010 (c) 1110 (d) 0111

8. The number of characters in standard ASCII is —

(a) 64 (b) 128 (c) 256 (d) 512

9. A single parity bit can —

(a) correct one error (b) detect any single (odd) error (c) detect two errors (d) correct two errors

10. By De Morgan, $\overline{(\bar{A} + \bar{B})}$ equals —

(a) A + B (b) $\bar{A}\cdot\bar{B}$ (c) A·B (d) $\bar{A} + \bar{B}$

11. Simplify $A + \bar{A}B$ —

(a) A (b) A + B (c) AB (d) B

12. The dual of A·(B + C) = AB + AC is —

(a) A + BC = (A+B)(A+C) (b) A(B+C) = A (c) $\bar{A}(\bar{B}+\bar{C})$ (d) none

13. F(A,B,C) = Σm(0, 2, 4, 6) minimises to —

(a) C (b) $\bar{C}$ (c) $\bar{A}$ (d) $A\bar{B}$

14. In a 4-variable K-map, a group of 8 cells yields a term with —

(a) 1 literal (b) 2 literals (c) 3 literals (d) 4 literals

15. A NAND gate with both inputs joined together acts as —

(a) AND (b) OR (c) NOT (d) buffer

16. Minimum number of NAND gates to build an XOR gate —

(a) 3 (b) 4 (c) 5 (d) 6

17. IC 7402 contains —

(a) quad NAND (b) quad NOR (c) hex NOT (d) quad XOR

18. Which logic family has the highest speed?

(a) TTL (b) CMOS (c) ECL (d) RTL

19. Which family has the lowest static power dissipation?

(a) TTL (b) ECL (c) CMOS (d) DTL

20. The sum output of a full adder with inputs A = 1, B = 1, Cin = 1 is —

(a) 0 (b) 1 with carry 0 (c) 1 with carry 1 (d) 0 with carry 0

21. The borrow output of a half subtractor (A − B) is —

(a) AB (b) $\bar{A}B$ (c) $A\bar{B}$ (d) $A \oplus B$

22. Select lines needed for a 16:1 multiplexer —

(a) 2 (b) 3 (c) 4 (d) 16

23. A DEMUX with 4 select lines has how many outputs?

(a) 4 (b) 8 (c) 16 (d) 32

24. In a JK flip-flop with J = K = 1, each clock pulse causes —

(a) set (b) reset (c) hold (d) toggle

25. The race-around condition is eliminated by —

(a) increasing pulse width (b) master–slave configuration (c) removing the clock (d) J = K = 0 always

26. Flip-flops needed for a MOD-100 counter —

(a) 6 (b) 7 (c) 8 (d) 10

27. A 5-stage Johnson counter has a MOD of —

(a) 5 (b) 10 (c) 25 (d) 32

28. A 4-bit ripple counter with FF delay 20 ns has f_max ≈ —

(a) 50 MHz (b) 25 MHz (c) 12.5 MHz (d) 6.25 MHz

29. An 8-bit DAC with 5 V full scale has step size ≈ —

(a) 19.6 mV (b) 39.2 mV (c) 9.8 mV (d) 5 mV

30. Which ADC gives the best 50 Hz noise rejection?

(a) flash (b) SAR (c) dual-slope (d) counter type

Answer Key with one-line explanations

Q	Ans	Explanation
1	b	$2 \cdot 256 + 10 \cdot 16 + 9 = 512 + 160 + 9 = 681$
2	a	110 101 011 → 6 5 3
3	b	$-2^7 \ldots +2^7-1 = -128 \ldots +127$
4	b	Invert every bit: 0110 → 1001
5	a	0101 + 1101 = 1 0010, drop carry → 0010 (5 − 3 = 2)
6	c	2421 is weighted **and** self-complementing; XS-3 is self-complementing but unweighted
7	a	Keep 1; $1 \oplus 1 = 0$; $1 \oplus 0 = 1$; $0 \oplus 1 = 1$ → 1011
8	b	7 bits → $2^7 = 128$
9	b	Parity detects any odd number of errors; corrects none
10	c	Complement of a sum of complements = AB
11	b	Redundancy law: $A + \bar{A}B = A + B$
12	a	Swap + and ·: the other distributive law
13	b	All even minterms have C = 0 → $F = \bar{C}$
14	a	n − k = 4 − 3 = 1 literal
15	c	$\overline{(A \cdot A)} = \bar{A}$
16	b	Standard 4-NAND XOR network
17	b	7402 = quad 2-input NOR
18	c	ECL never saturates → ~1 ns delays
19	c	CMOS draws only leakage current statically
20	c	$1+1+1 = 11_2$ → Sum 1, Carry 1
21	b	Borrow when A = 0, B = 1 → $\bar{A}B$
22	c	$\log_2 16 = 4$
23	c	$2^4 = 16$ outputs
24	d	J = K = 1 is the toggle mode
25	b	Master–slave (or edge triggering) allows one toggle per pulse
26	b	$2^7 = 128 \geq 100 > 64 = 2^6$ → 7 FFs
27	b	Johnson MOD = 2n = 10
28	c	1/(4 × 20 ns) = 12.5 MHz
29	a	$5/(2^8 - 1) = 5/255 \approx 19.6$ mV
30	c	Dual-slope integrates over a mains period → hum cancels

3.15 Rapid-Revision Sheet

Flip-flop master table

FF	Truth table (inputs → Q^+)	Characteristic eqn	Excitation ($Q \to Q^+$: inputs)
SR	00→Q, 01→0, 10→1, 11→invalid	$Q^+ = S + \bar{R}Q$	0→0: 0×; 0→1: 10; 1→0: 01; 1→1: ×0
JK	00→Q, 01→0, 10→1, 11→**toggle**	$Q^+ = J\bar{Q} + \bar{K}Q$	0→0: 0×; 0→1: 1×; 1→0: ×1; 1→1: ×0
D	$Q^+ = D$	$Q^+ = D$	D = desired Q^+
T	0→hold, 1→**toggle**	$Q^+ = T \oplus Q$	same: 0; change: 1

Conversions: JK→T (J=K=T), JK→D (J=D, K=$\bar{D}$), D→T (D = T⊕Q), T→D (T = D⊕Q).

Counter formulas

- FFs for MOD-N: **$n = \lceil \log_2 N \rceil$**; ripple/sync binary MOD = 2^n; **ring = n; Johnson = 2n**.
- Frequency division: n cascaded toggle stages → **$f/2^n$**; T = 1 FF alone → f/2.
- Ripple counter f_{max} = **$1/(n \cdot t_{pd})$**; synchronous $f_{max} = 1/(t_{pd} + t_{gate})$.
- Count after P pulses (from 0) = **P mod 2^n** in binary.
- Decade counter (7490): resets at 1010 via NAND(Q_3, Q_1); counts 0–9.

ADC / DAC formulas

- **DAC step size = $V_{FS}/(2^n - 1)$**; V_{out} = D × step; % resolution = $100/(2^n - 1)$.
- **ADC resolution Q = $V_{FS}/2^n$**; quantisation error = **±½ LSB**; SNR ≈ 6.02n + 1.76 dB.
- Flash: **$2^n - 1$ comparators**, fastest. SAR: **n clocks**, fixed. Counter: ≤ $2^n - 1$ clocks. Dual-slope: $V_{in} = V_{ref} \cdot T_2/T_1$, most accurate, slowest.
- R-2R DAC: only two resistor values; weighted-resistor DAC: spread 2^{n-1}:1 (impractical for large n).

Number-system nuggets

- n bits → 2^n values; signed 2's-comp range $-2^{n-1} \ldots 2^{n-1} - 1$.
- 2's complement shortcut: copy up to first 1 from LSB, invert the rest.
- Binary→Gray: $G_i = B_{i+1} \oplus B_i$. XS-3 = BCD + 3. BCD fix-up: add 6 if group > 9 or carries.
- Hamming: $2^k \geq m + k + 1$; distance d → detect d−1, correct ⌊(d−1)/2⌋.

Gate one-liners

- XOR: 1 iff inputs differ; **1 ⊕ 1 = 0**; odd-parity function; A ⊕ 1 = $\bar{A}$.
- Universal gates: NAND, NOR. XOR from NANDs: **4**. Bubbled AND = NOR; bubbled OR = NAND.
- HA: S = A⊕B, **C = AB**. FA: S = A⊕B⊕Cin, Cout = AB + BCin + ACin. HS borrow = **$\bar{A}B$**.
- Decoder n→2^n: exactly **1 active output**. MUX N:1 needs $\log_2 N$ selects. Priority encoder: **highest active input wins**.

Logic-family comparison (memorise the extremes)

Property	Winner
Fastest (lowest t_pd)	**ECL** (~1 ns; never saturates)
Lowest static power	**CMOS** (~nW)
Highest power drain	ECL
Highest fan-out / noise immunity	**CMOS** (≥50; margins ~0.3–0.45 V_DD)
Standard supply	TTL: +5 V; ECL: −5.2 V; CMOS: 3–15 V
TTL levels	V_IL 0.8, V_IH 2.0, V_OL 0.4, V_OH 2.4 → NM = 0.4 V

Memory one-liners

- **SRAM**: 6-T latch, no refresh, fast, cache. **DRAM**: 1T-1C, needs refresh, dense/cheap, main memory. Both volatile.
- EPROM erased by **UV (whole chip)**; EEPROM **electrically (byte)**; Flash **electrically (block)**; all non-volatile floating-gate.
- M-word memory needs **$\log_2 M$ address lines**; chips for A×B from a×b = (A/a)×(B/b).
- PROM: fixed AND + prog OR; PAL: prog AND + fixed OR; PLA: both programmable.
- Hierarchy: registers → cache (SRAM) → main (DRAM) → flash/disk; t_avg = h·t_c + (1−h)·t_m.

End of Chapter 3. Next: Chapter 4 — Linear Integrated Circuits.